Mathematisch-Naturwissenschaftliche Fakultät

Lernbasierte Computer Vision

Masterarbeit

LLM-based Academic Writing with Fine-Grained Citations

Eberhard Karls Universität Tübingen
Mathematisch-Naturwissenschaftliche Fakultät
Wilhelm-Schickard-Institut für Informatik
Lernbasierte Computer Vision
Niklas Munkes, niklas.munkes@student.uni-tuebingen.de, 2025

Bearbeitungszeitraum: 14.04.-14.10.2025

Gutachter: Prof. Dr. Andreas Geiger, Universität Tübingen
Zweitgutachter: Prof. Dr. Carsten Eickhoff, Universität Tübingen
Betreuer: Haoyu He, Universität Tübingen

Selbstständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Masterarbeit selbständig und nur mit den angegebenen Hilfsmitteln angefertigt habe und dass alle Stellen, die dem Wortlaut oder dem Sinne nach anderen Werken entnommen sind, durch Angaben von Quellen als Entlehnung kenntlich gemacht worden sind. Diese Masterarbeit wurde in gleicher oder ähnlicher Form in keinem anderen Studiengang als Prüfungsleistung vorgelegt.

Niklas Munkes (Matrikelnummer 4269436), September 20, 2025

Abstract

TODO

Contents

1. Introduction

TODO

citations build trust [DOW+24]

Retrieval-Augmented Generation (RAG, [LPP+20]) models are designed to generate text that is grounded in real-world factual knowlege. They have been widely adopted for knowledge-intensive NLP tasks [SQK+25] such as long-form question answering [XWL+24], scientific question answering [LOS+23] and question synthesis [XLS+24], generation with sentence-level citations [ZBL+24, XWL+24] and academic text generation [AHS+24, WMN+25]. RAG models usually consist of a generator, a pre-trained sequence-to-sequence (seq2seq) model (e.g. a Transformer-based [VSP+17] decoder), and a retriever with access to a retrieval corpus, which may be implemented as a dense vector index (e.g. a HNSW [MY18] index) [LPP+20]. The RAG architecture is briefly introduced in section 3.1. Such models combine pre-trained paramentric memory with non-paramentric (retrieval-based) memory [LPP+20], which has a number of benefits: while neural language models are able to store a substantial amount of world knowledge in their parameters [RRS20], this knowledge can't easily be modified, expanded or directly be used to gain insight into their decision-making process [LLG+19]. By using an external retrieval corpus, the knowledge RAG models use to inform their generation can directly be inspected and interpreted [LPP+20]. Their TODO

TODO

TODO

2. Related Work

TODO

2.1. LLM-based Generation

[STB25] "Using human experts to evaluate the quality of LLM answers is generally considered the gold standard, but expert annotation is costly and slow" [HPS+25]
TODO

2.2. RAG for Scientific Writing

Previous work on text generation with citations has recently transitioned from treating retrieval and generation as seperate processing stages (e.g. [SHC+24]) to alternating between the two (e.g. [XWL+24], [AHS+24]). *Attribute First, then Generate* [SHC+24] initially retrieves a set of relevant sub-sentence spans which are then grouped into coherent clusters which in turn are used to inform an iterative sentence-by-sentence generation step [SHC+24]. *ReClaim* [XWL+24], more specifically the *ReClaim with Interleaving Generation* method, is used to generate an output sequence $O = \{r_1, c_1, r_2, c_2, ..., r_n, c_n\}$ of alternating references and claims where claim c_i is the portion of the models response that was generated based on reference r_i. Claim and reference generation is performed by seperate LLMs specifically trained for their corresponding task [XWL+24]. In contrast, *OpenScholar* [AHS+24] first retrieves and reranks a set of papers P relevant to the given query. These papers inform a subsequent generation step that produces a generated response r_i [AHS+24]. This response is refined through iterative *self-feedback* where during each iteration of the feedback loop $F = f_1, f_2, ..., f_T$, the generator model generates sentences f_j that describe potential improvements to the response and may also prompt the retrieval pipeline to retrieve additional papers, should r_i require more information to generate an improved response r_{i+1} [AHS+24]. Depending on the size of P and the choice of T, OpenScholar still performs retrieval and generation largely seperately.

ScholarCopilot [WMN+25] is a RAG system developed for generating academic-style text with accurate citations. It is intended as a tool that assists the user with academic paper writing by providing iterative text generation with automated citation retrieval [WMN+25]. ScholarCopilot also enables user interaction during

generation [WMN+25] by allowing the user to rewrite the generated text or forcefully trigger citation retrieval between the consecutive generation steps but this feature is beyond the scope of this thesis and is thus not utilized for the experiments. ...

3. Methods

TODO

3.1. Retrieval-Augmented Generation

The RAG architecture [LPP+20] is comprised of a retriever p_η consisting of a query encoder q and a retrieval corpus d, and a generator p_θ (see Figure 3.1). Given a query x, the retriever $p_\eta(z|x)$ (with parameters η) computes a distribution over the text documents z in the retrieval corpus d according to x [LPP+20]. This distribution is used to obtain the top-k most relevant documents with respect to the query. The generator $p_\theta(y_i|x, z, y_{1:i-1})$, parametrized by θ, then predicts each next token y_i based on the previous tokens $y_{1:i-1}$, the given query x and a document z from the top-k retrieved documents (according to p_η) [LPP+20]. Lewis et al. base their retriever on the `Dense Passage Retriever` [KOM+20] and use a pre-trained `BART-large` [LLG+19] as the generator.

3.2. Dataset

This thesis evaluates the retrieval performance and generation quality of ScholarCopilot (SC, see section 3.3) with and without the passage retrieval module (PR, see section 3.4). ScholarCopilot retains its original retrieval corpus, while the retrieval corpus for the PR module uses the HDT dataset [HFB+24], since it requires fulltext data and the SC retrieval corpus only consists of abstracts. The PR module constructs the passages for each individual document during inference. The datasets used for evaluation are constructed from the HDT dataset as well.

The datasets for retrieval evaluation (section 4.2) each use a subset of the HDT dataset. Each test sample in the dataset consists of a citing text span and the corresponding cited document. Following the evaluation setup in [WMN+25], all tokens after the citation are removed. This emulates the context the model would receive during generation, where it also only has access to the left-hand context of each `[RET]` token (see section 3.3, particularly the definition of $\hat{c}_j$ following Eq. 3.2). The citing context is grouped into separate datasets based on which section in the source document it is drawn from. This results in four datasets comprised of context from the sections `IntroductionAndRelatedWork`, `Methods`, `Experiments` and `Conclusion`.

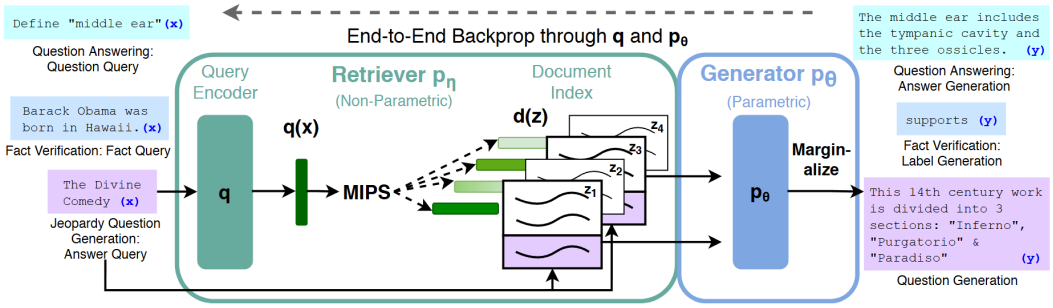

Figure 3.1.: Overview of the RAG architecture initially proposed by [LPP+20]. For finding the top-k documents Z', the authors used Maximum Inner Product Search (MIPS).

The candidate sections in the HDT dataset for each of these datasets are determined via exact token matching, e.g. if the lowercase title of a section contains the string "methods", it is considered a methods section and therefor used to extract samples for the `Methods` evaluation dataset. Note that the dataset `IntroductionAndRelatedWork` combines two kinds of sections commonly found in scientific documents, this is done to imitate the ScholarCopilot evaluation dataset that includes only these sections. ScholarCopilot should yield the best performance on this dataset since the model is trained exclusively on titles, abstracts, introductions and related work sections [WMN+25].

The dataset for the generation evaluation (section 4.3) also uses the HDT dataset. Specifically, each sample in the dataset consists of the title, abstract and the first two and a half sentences of the introduction (the HDT dataset already provides sentences, the half-sentence is obtained by tokenizing the source sentence using the ScholarCopilot tokenizer and truncating it to its left-hand half based its total number of tokens). This setup mimics the examples for initial generation contexts[1] provided with the official ScholarCopilot implementation. Before being used for evaluation, all samples are sorted by the arXiv id of the paper they were obtained from, which effectively also sorts them by their month of publication (see [sta25b]), and only the k most recently published papers are retained. This is done to ensure that ScholarCopilot predominantly cites papers published after the paper it is prompted to generate (that is, the paper that is used as the initial generation context).

3.3. ScholarCopilot

Instead of treating retrieval and generation as separate processing stages (see section 1) ScholarCopilot [WMN+25] integrates the information retrieval directly into the generation process. The model is trained to generate *retrieval tokens* (`[RET]`), that upon

[1] `https://github.com/TIGER-AI-Lab/ScholarCopilot/tree/main/run_demo/src`

being generated, cause the model to interupt the generation process to retrieve a reference from the retrieval corpus [WMN+25]. The ScholarCopilot retrieval corpus[2] consists of abstracts of scientific papers, which are encoded by the ScholarCopilot model prior to training/inference and stored as a HNSW [MY18] index using Faiss [DGD+24] for efficient retrieval. Standard ScholarCopilot uses the retrieved abstract to inform its subsequent generation by replacing the `[RET]` tokens in the generated context with the corresponding abstracts.

For this thesis, I evaluated the official ScholarCopilot implementation[3]. In their paper, the authors state that ScholarCopilot can also integrate key excerpts[4] [WMN+25] but the official implementation only uses abstracts and provides no logic for extracting these excerpts. Because of that, I further on use the term "ScholarCopilot" to denote a version of this model that represents papers by their abstracts and does not use potentially more informative key excerpts.

ScholarCopilot is intended to assist with academic writing by providing "[...] text completion and citation suggestions" (see the official demo[5,6] for an example). In the following we will thus assume that the model is used to generate a scientific paper with citations. Let

$$Y^{\mathtt{in}} = \left(y_1^{\mathtt{in}}, y_2^{\mathtt{in}}, ..., y_l^{\mathtt{in}}\right) \tag{3.1}$$

denote the initial context given to the model. Since we seek to generate a scientific document, this context may consist of the title and abstract of the paper the model is supposed to generate. Let furthermore

$$Y_{1:i-1}^{\mathtt{gen}} = \left(y_1^{\mathtt{gen}}, y_2^{\mathtt{gen}}, ..., y_{j-1}^{\mathtt{gen}}, c_j, y_{j+1}^{\mathtt{gen}}, ..., y_{i-1}^{\mathtt{gen}}\right) \tag{3.2}$$

denote a token sequence generated by the model where c_j signifies that a `[RET]` token was generated by the model at generation step j. Note that the model may generate the retrieval token at multiple steps, and that the number of generated retrieval tokens is equal to the number of citations in the generated paper. The representation $\hat{c}_j$ of a `[RET]` token c_j computed by the model serves as an aggregated representation of the token sequence generated up to the `[RET]` token (e.g. $Y_{1:j-1}^{\mathtt{gen}}$ in Eq. 3.2), in the same manner as BERT [DCLT19] (and also for example CF-ViT [CLL+23]) uses the `[cls]` token as a representation of its input sequence. Thus $\hat{c}_j$ is unique for every j since it is dependent of the corresponding left-hand context.

When the ScholarCopilot generator p_G generates a retrieval token (that is, $p_G(*|Y^{\mathtt{in}}Y_{1:i-1}^{\mathtt{gen}})$ is maximal for the `[RET]` token, thus $* \leftarrow c_i$), the generation process is interrupted

[2] `https://huggingface.co/datasets/TIGER-Lab/ScholarCopilot-Data-v1`

[3] See `https://github.com/TIGER-AI-Lab/ScholarCopilot`. I cloned the repository on the 13th of June 2025 and evaluated that version, which corresponds to commit `f8e6de9` in the source repository. As of the time of writing this (7th of September), only the README.md was extended.

[4] In the paper it says "key excepts", likely a typo.

[5] `https://huggingface.co/spaces/TIGER-Lab/ScholarCopilot`

[6] `https://www.youtube.com/watch?v=QlY7S52sWDA`

Algorithm 1: Overview of one generation step of ScholarCopilot. Here Y denotes the vocabulary (the set of all tokens), Z the retrieval corpus (all encoded abstracts), G ScholarCopilot's decoder and getAbstract() a function that returns the unencoded abstract corresponding to z_i

Input: The initial context $Y^{\texttt{in}}$ and the previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$
Output: The initial context $Y^{\texttt{in}}$ and the new generated sequence $Y^{\texttt{gen}}_{1:i}$

Algorithm: ScholarCopilotGenerationStep($Y^{\texttt{in}}, Y^{\texttt{gen}}_{1:i-1}$)
$y_i \leftarrow \arg\max_{y_i \in Y} p_G(y_i | Y^{\texttt{in}} Y^{\texttt{gen}}_{1:i-1})$
if y_i = *[RET]* **then**
  $\hat{c}_i \leftarrow G(y_i)$
  $z_i \leftarrow \arg\max_{z_i \in Z} p_R(z_i | \hat{c}_i)$
  $Y^{\texttt{ref}}_i \leftarrow$ getAbstract(z_i)
  $\mathbb{Y}^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} Y^{\texttt{ref}}_i$
  return Y^{in}, $\mathbb{Y}^{gen}_{1:i}$
end
else
  $Y^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} y_i$
  return Y^{in}, $Y^{gen}_{1:i}$
end

and instead, the representation $\hat{c}_i$ is passed to the retriever $p_R(z_i|\hat{c}_i)$ used to retrieve an encoded reference z_i corresponding to an abstract

$$Y^{\texttt{ref}}_i = \left(y^{\texttt{ref}}_{i,1}, y^{\texttt{ref}}_{i,2}, ..., y^{\texttt{ref}}_{i,k}\right) \tag{3.3}$$

from the retrieval corpus. Before passing the newly generated context to the generator the retrieval token c_i is replaced with the corresponding abstract to obtain an augmented generation output

$$\mathbb{Y}^{\texttt{gen}}_{1:i} = \left(y^{\texttt{gen}}_1, y^{\texttt{gen}}_2, ..., y^{\texttt{gen}}_{i-1}, y^{\texttt{ref}}_{i,1}, y^{\texttt{ref}}_{i,2}, ..., y^{\texttt{ref}}_{i,k}\right) \tag{3.4}$$

that is used to inform the sebsequent generation steps with the retrieved information. The generator $p_G(y_{i+1}|Y^{\texttt{in}}\mathbb{Y}^{\texttt{gen}}_{1:i})$ predicts the next token y_{i+1} at generation step $i+1$ based on the given initial context $Y^{\texttt{in}}$ and the augmented output $\mathbb{Y}^{\texttt{gen}}_{1:i}$. See Algo. 1 for a detailed overview of one generation step. Before presenting the generated sequence to the user, each inserted abstract is replaced with a token referencing the cited paper (when using latex formatting, this would for example be a citation like "\cite{vaswani2017attention}"). The user is presented with this modified sequence, as well as a list of all cited references.

3.4. Passage Retrieval

As outlined in section 3.3, ScholarCopilot (SC) represents each scientific document in the retrieval corpus with its abstract. But, depending on the context for which the model is supposed to retrieve a reference, the abstract might not be the most informative passage of a candidate paper (see section 1). Thus always using a paper's abstract as reference may add irrelevant information into the generated sequence which can negatively affect the model's reasoning capabilities [SCM+23].

This thesis introduces a *passage retrieval* module (PR) that is supposed to enhance the citation accuracy and generation quality of ScholarCopilot (see section 1). The module is agnostic to the RAG model, which makes it possible to use with any RAG architecture. However, this thesis only investigates how it affects ScholarCopilot. First, the module takes the top-k_{SC} documents

$$Z_i = \left(D_1, D_2, ..., D_{k_{SC}}\right) \tag{3.5}$$

returned by the SC retriever for a previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$ and subdivides these documents into n-sized passages to obtain a set of candidate passages P_i.

Prior work on understanding and utilizing discourse structure of prose has shown that the grammatical structure across multiple sentences can inform the reader of what is important in a document [Rum75] and can guide comprehension and recall [Man78]. Since the PR retrieval corpus (see section 3.2) consists of documents published on arXiv[7] and thus they are curated and "[checked] for scholarly value" [sta25a], it is reasonable to assume that these documents are already well-formed and have meaningful semantic structure spanning multiple sentences and paragraphs. To preserve this structure of each document D_i in the passages obtained from it as much as possible, each document D_i is first split into sections

$$D_i = \left(\mathcal{E}_1, \mathcal{E}_2, ..., \mathcal{E}_{|D|}\right) \tag{3.6}$$

by utilizing the fact that the HDT dataset [HFB+24] (see also section 3.2) already provides its documents structured into sections. Each section $\mathcal{E}_i$ is split into passages

$$\mathcal{E}_i = \left(\mathcal{P}_1, \mathcal{P}_2, ..., \mathcal{P}_{|\mathcal{E}_i|}\right) \tag{3.7}$$

which are sequences of n or less tokens $\mathcal{P}_i = (t_1, t_2, ..., t_n)$ ($\mathcal{P}_{|\mathcal{E}_i|}$ might be shorter than n tokens since $|\mathcal{E}_i| \equiv 0 \mod n$ likely does not hold for every section in every document). This ensures that each passage only consists of sentences taken from a single section in the source document. All passages are collected into a corpus P_i and then ranked by a decoder model R_{PR} based on how relevant they are to the context $Y^{\texttt{gen}}_{1:i-1}$ (assuming y_i denotes the `[RET]` token that triggered the retrieval, see section 3.3, particularly Algo. 1). To achieve reliably reproducable relevance assessment performance the

[7] `https://arxiv.org/`

scores computed by R_{PR} are aggregated using Batched Self-Consistency [KDSR25]. Furthermore, to reduce the computational burden, the context is truncated to size w – yielding $Y^{\texttt{gen}}_{i-w-1:i-1}$ as the new context – before being passed to R_{PR}. I found that passing longform context tends to worsen the performance of the passage ranking model. The top-ranked passage is then integrated into the generated text in the same manner as the abstract would have been integrated when using standard ScholarCopilot (see Eq. 3.4). How the PR module is integrated into ScholarCopilot is shown in Algo. 2.

The decision to let the ScholarCopilot retriever first retrieve abstracts and not directly passages is motivated by the success of employing mixed-resolution tokenization [CLL+23, HRBB23] for image processing with the Vision Transformer [DBK+20]. In [CLL+23, HRBB23], mixed-scale tokenization is primarily used to decrease the length of the input sequence to alleviate the computational burden imposed by the Transformer backbone whose attention layer has $O(n^2)$ time complexity (see [VSP+17]). Both approaches first split a given image into larger *coarse-scale* patches and then identify the most informative k patches which are then further subdivided into four *fine-scale* tokens before the image, now represented by a mixture of coarse and fine-scale tokens, is passed to the ViT backbone. ScholarCopilot and the PR retriever use the same backbone architecture (see section 4) for retrieval, so if we consider the abstract of a document D_i its coarse-scale representation and its $|D_i|$ passages (for example, when assuming 8192 tokens per document and a passage length of 512 this would mean $|D_i| = 16$) to be the fine-scale representation, then the PR module implements a similar two-stage *mixed-scale retrieval*. Let C denote the retrieval corpus and for simplicity lets assume that all documents $D_i \in C$ have the same number of $|D_i|$ passages. Then performing passage retrieval on the set of all passages would mean comparing one query to $|C| \cdot |D_i|$ passages. Using mixed-scale retrieval would result in $|C| + k_{SC}|D_i|$ comparisons, a lot less when k_{SC} is chosen significantly smaller that $|C|$ (which is reasonable to assume and is also the case for my experiments, see section A.1). This effect is further amplified when using Batched Self-Consistency.

The abstract of a scientific document typically presents the reader with knowlege of its content, can arouse interest and can indicate whether the fulltext or part of it merits further attention [SM90]. It can serve as a condensed representation of the whole document [CO06]. An abstract can thus also be considered a semantic coarse-scale representation of a document since it usually contains short general statements about the topics discussed in the document. For example, the sentence "We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely." from the abstract of [VSP+17] conveys the information that the Transformer is based on attention but does not elaborate on how attention works.

On the other hand, the passages obtained from a document's sections often contain more specific information. Consider for example "We call our particular attention

'Scaled Dot-Product Attention' [...]. The input consists of queries and keys of dimension [...]" (the beginning of section 3.2.1 in [VSP+17]) which leads into a detailed explanation of the attention layer. Passages can thus similarly be considered *fine-scale* in a semantic sense, which allows the PR retriever to better distinguish between different aspects of the topic discussed in each paper than ScholarCopilot (that relies on abstracts alone). This could potentially result in increased retrieval performance, especially if the PR retriever is specifically trained with a contrastive loss function to identify relevant passages. However, this remains an assumption for now because training the retriever is beyond the scope of this thesis and is thus left for future research (see also section 5).

Algorithm 2: Overview of one generation step of ScholarCopilot with Passage Retrieval. getDocument() denotes a function that returns the fulltext paper corresponding to z_j

Input: The initial context $Y^{\texttt{in}}$ and the previously generated sequence $Y^{\texttt{gen}}_{1:i-1}$
Output: The initial context $Y^{\texttt{in}}$ and the new generated sequence $Y^{\texttt{gen}}_{1:i}$

Algorithm: ScholarCopilotWithPRGenerationStep($Y^{\texttt{in}}, Y^{\texttt{gen}}_{1:i-1}$)
$y_i \leftarrow \arg\max_{y_i \in Y} p_G(y_i | Y^{\texttt{in}} Y^{\texttt{gen}}_{1:i-1})$
if y_i = *[RET]* **then**
  $\hat{c}_i \leftarrow G(y_i)$
  $Z_i \leftarrow \emptyset$
  $Z' \leftarrow Z$
  while $|Z_i| < k_{SC}$ **do**
    $z_j \leftarrow \arg\max_{z_j \in Z'} p_R(z_j | \hat{c}_i)$
    $Z_i \leftarrow Z_i \cup \{z_j\}$
    $Z' \leftarrow Z' \setminus \{z_j\}$
  end
  $P_i \leftarrow \emptyset$
  foreach $z_j \in Z_i$ **do**
    $D_j \leftarrow$ getDocument(z_j)
    foreach $\mathcal{P}_k \in D_j$ **do**
      $P_i \leftarrow P_i \cup \mathcal{P}_k$ // see Eq. 3.6 and Eq. 3.7
    end
  end
  $Y^{\texttt{ref}}_i \leftarrow \arg\max_{\mathcal{P}_k \in P_i} p_{R_{PR}}\left(\mathcal{P}_k | Y^{\texttt{gen}}_{i-w-1:i-1}\right)$
  $\mathbb{Y}^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} Y^{\texttt{ref}}_i$
  return Y^{in}, $\mathbb{Y}^{gen}_{1:i}$
end
else
  $Y^{\texttt{gen}}_{1:i} \leftarrow Y^{\texttt{gen}}_{1:i-1} y_i$
  return Y^{in}, $Y^{gen}_{1:i}$
end

4. Experiments

4.1. Evaluation Methodology

The main focus of this thesis is it to determine how the use of the passage retrieval module affects the retrieval accuracy and generation quality of ScholarCopilot. ScholarCopilot retains its original backbone model `Qwen2.5-7B` [YYZ+24]. Since the retriever of the passage retrieval module is implemented with Batched Self-Consistency, it is required to follow multiple permutations of the same instruction during inference reliably and consistently (see [KDSR25]). Instruction tuned models, that is, LLMs that are fine-tuned on (`instruction`, `desired-output`) pairs, are more successful at following instructions closely rather than just minimizing the next token prediction error [ZDL+25]. To make use of this, the PR module uses the instruction-tuned variant of the Qwen2.5 model, `Qwen2.5-7B-Instruct`, as the passage retriever. Even though it has been shown that instruction tuned models may not fully utilize given instructions and are instead just good at learning the specified output format [KP23], this ability alone is valuable enough for the automated parsing of an LLMs' response (especially when using Batched Self-Consistency which greatly increases the number of responses that need to be parsed) to justify using an instruction tuned model.

TODO

4.2. Single-Step Retrieval

With ScholarCopilot as baseline, I evaluated the impact of using the PR module on the retrieval performance for a variety of retrieval contexts. To this end, I introduce the *single-step retrieval* task which aims to assess the models accuracy when resolving a single `[RET]` token during generation. This task closely follows the evaluation methodology used by [WMN+25]. Single-step retrieval is evaluated using the Recall$k$ metric, which is given as the fraction of cases where the ground truth document appears in the top-k retrieved documents [WMN+25]. As outlined in section 3.2, the model is given the left-hand context of a citation drawn from an existing paper, together with the cited document as the ground truth. A retrieval token is appended to this context before the resulting sequence is passed to ScholarCopilot, causing the model to compute a representation of the `[RET]` token based on the given context. This representation is then used to compute a distribution over the ScholarCopilot

retrieval corpus that measures the relevance of each document in the corpus to the given context. When evaluating standart ScholarCopilot using Recall$k$, the k most relevant documents according to this distribution are compared to the ground truth, otherwise the top-k_{SC} most relevant documents are passed to the passage retrieval module (for the reported results I use $k_{SC} = 2k$, see also section A.1). The PR module returns for each of the top-k ranked passages the document from which passage was taken. These k documents are then compared to the provided ground truth. The given citing context is not constrained when passed to ScholarCopilot.[1] I found that passing the entire context to the PR module reduces performance, thus only the last w tokens are used.

The results of the single-step retrieval evaluation are visualized in Figure **??**. ScholarCopilot performs consistently better without the passage retrieval module (with an average increase of TODO). It is important to point out that the performance of the PR module is partially dependent on the performance of ScholarCopilot, due to the choice of $k_{SC} = 2k$. For Recall10 for example, it is impossible for the correct document to be chosen by the PR module if it is not in the top 20 documents ranked by ScholarCopilot. This effect can be mitigated by choosing a larger k_{SC}, but this would also drastically increase the effective runtime of the PR module. Let ...

4.3. Generation Quality

TODO

[1] [WMN+25] only use the last sentence of the context when evaluating their baseline models (`BM25` and `E5-Mistral-7B-Instruct`) but use the full context when evaluating ScholarCopilot.

5. Limitations and Future Work

TODO

6. Conclusion

TODO

A. Appendix

A.1. Hyperparameters

TODO

A.2. TODO

TODO

Bibliography

[AHS+24] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'arcy, David Wadden, Matt Latzke, Minyang Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Dan Weld, Doug Downey, Wen tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. Openscholar: Synthesizing scientific literature with retrieval-augmented lms. *arXiv preprint arXiv:2411.14199*, 2024.

[CLL+23] Mengzhao Chen, Mingbao Lin, Ke Li, Yunhang Shen, Yongjian Wu, Fei Chao, and Rongrong Ji. Cf-vit: A general coarse-to-fine method for vision transformer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 7042–7052, 2023.

[CO06] Cate Cross and Charles Oppenheim. A genre analysis of scientific abstracts. *Journal of Documentation*, 62:428–446, 07 2006.

[DBK+20] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[DGD+24] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.

[DOW+24] Hyo Jin Do, Rachel Ostrand, Justin D Weisz, Casey Dugan, Prasanna Sattigeri, Dennis Wei, Keerthiram Murugesan, and Werner Geyer. Facilitating human-llm collaboration through factuality scores and source attributions. *arXiv preprint arXiv:2405.20434*, 2024.

[HFB+24] Haoyu He, Markus Flicke, Jan Buchman, Iryna Gurevych, and Andreas

Geiger. HDT: Hierarchical Document Transformer. Conference on Language Modeling, 2024.

[HPS+25] Sebastian Heineking, Jonas Probst, Daniel Steinbach, Martin Potthast, and Harrisen Scells. Ranking generated answers: On the agreement of retrieval models with humans on consumer health questions. In *European Conference on Information Retrieval*, pages 128–137. Springer, 2025.

[HRBB23] Jakob Drachmann Havtorn, Amélie Royer, Tijmen Blankevoort, and Babak Ehteshami Bejnordi. Msvit: Dynamic mixed-scale tokenization for vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 838–848, 2023.

[KDSR25] Anton Korikov, Pan Du, Scott Sanner, and Navid Rekabsaz. Batched self-consistency improves llm relevance assessment and ranking. *arXiv preprint arXiv:2505.12570*, 2025.

[KOM+20] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pages 6769–6781, 2020.

[KP23] Po-Nien Kung and Nanyun Peng. Do models really learn to follow instructions? an empirical study of instruction tuning. *arXiv preprint arXiv:2305.11383*, 2023.

[LLG+19] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.

[LOS+23] Jakub Lála, Odhran O'Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G Rodriques, and Andrew D White. Paperqa: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2023.

[LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[Man78] Jean M. Mandler. A code in the node: The use of a story schema in retrieval. *Discourse Processes*, 1(1):14–35, 1978.

[MY18] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs.

IEEE transactions on pattern analysis and machine intelligence, 42(4):824–836, 2018.

[RRS20] Adam Roberts, Colin Raffel, and Noam Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

[Rum75] David E. Rumelhart. Notes on a schema for stories. In Daniel G. Bobrow and Allan Collins, editors, *Representation and Understanding*, pages 211–236. Morgan Kaufmann, San Diego, 1975.

[SCM+23] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *International Conference on Machine Learning*, pages 31210–31227. PMLR, 2023.

[SHC+24] Aviv Slobodkin, Eran Hirsch, Arie Cattan, Tal Schuster, and Ido Dagan. Attribute first, then generate: Locally-attributable grounded text generation. *arXiv preprint arXiv:2403.17104*, 2024.

[SM90] Françoise Salager-Meyer. Discoursal flaws in medical english abstracts: A genre analysis per research- and text-type. *Text - Interdisciplinary Journal for the Study of Discourse*, 10, 01 1990.

[SQK+25] Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen-tau Yih, Pang Wei Koh, and Luke Zettlemoyer. Reasonir: Training retrievers for reasoning tasks. *arXiv preprint arXiv:2504.20595*, 2025.

[sta25a] arXiv staff. About arXiv, 2025. `https://info.arxiv.org/about/index.html`, Last accessed: 10.09.2025.

[sta25b] arXiv staff. Understanding the arXiv identifier, 2025. `https://info.arxiv.org/help/arxiv_identifier.html`, Last accessed: 15.09.2025.

[STB25] Shubham Sharma, Sneha Tuli, and Narendra Badam. Challenges and applications of large language models: A comparison of gpt and deepseek family of models. *arXiv preprint arXiv:2508.21377*, 2025.

[VSP+17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[WMN+25] Yubo Wang, Xueguang Ma, Ping Nie, Huaye Zeng, Zhiheng Lyu, Yuxuan Zhang, Benjamin Schneider, Yi Lu, Xiang Yue, and Wenhu Chen. Scholarcopilot: Training large language models for academic writing with accurate citations. *arXiv preprint arXiv:2504.00824*, 2025.

[XLS+24] Fangyuan Xu, Kyle Lo, Luca Soldaini, Bailey Kuehl, Eunsol Choi, and David Wadden. Kiwi: A dataset of knowledge-intensive writing instructions for answering research questions. *arXiv preprint arXiv:2403.03866*, 2024.

[XWL+24] Sirui Xia, Xintao Wang, Jiaqing Liang, Yifei Zhang, Weikang Zhou, Jiaji Deng, Fei Yu, and Yanghua Xiao. Ground every sentence: Improving retrieval-augmented llms with interleaved reference-claim generation. *arXiv preprint arXiv:2407.01796*, 2024.

[YYZ+24] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[ZBL+24] Jiajie Zhang, Yushi Bai, Xin Lv, Wanjun Gu, Danqing Liu, Minhao Zou, Shulin Cao, Lei Hou, Yuxiao Dong, Ling Feng, and Juanzi Li. Longcite: Enabling llms to generate fine-grained citations in long-context qa. *arXiv preprint arXiv:2409.02897*, 2024.

[ZDL+25] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, and Guoyin Wang. Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*, 2025.